



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UTM Johor Bahru

Faculty of Computing
Semester II, Session
2025/26

SECJ3483 SEC01
WEB TECHNOLOGY

Individual Assignment & DHL DAC 3.0

Project Title:

**AI-Powered Knowledge Base Automation for DHL Logistics
Operations**

Name	Matric No
Yap Jia Xin	A23CS0199

Submission Date: 13/5/2026

Table of Contents

1. Introduction and Scenario Description.....	2
1.1 Project Background.....	2
2. System Architecture.....	2
3. API and JSON Structure.....	3
4. RPA Workflow and Automation Explanation.....	5
5. User Interface Screenshots with Description.....	6
6. User Manual.....	9

1. Introduction and Scenario Description

1.1 Project Background

In logistics companies like DHL, operational data is created every day in many different formats. These include PDF manuals, Word documents, PowerPoint slides, emails, chat messages, and scanned images. Workers often need to search through many files when they want to solve a problem or follow a Standard Operating Procedure (SOP). This process takes a lot of time and can cause mistakes because the information is not organized properly.

The DHL DAC 3.0 assignment focuses on Scenario 1, which is AI-Powered Knowledge Base Automation for DHL Logistics Operations. The main goal is to transform messy and unstructured operational information into clean and structured knowledge articles automatically.

To solve this problem, the AI-Powered Knowledge Base System was developed. The system works as a centralized platform for storing and managing logistics knowledge articles. Users and RPA bots can upload operational files into the system. The backend extracts the text from the uploaded files. Then the system uses Google Gemini AI to generate structured content such as SOP steps, summaries, warnings, and tags. The generated article is saved as a Draft first. After that, administrators or reviewers can check the content before publishing it to the system.

The system also uses a workflow process that contains Draft, Reviewed, and Published status. This workflow helps ensure that every article is checked before becoming visible to workers.

2. System Architecture

The project uses a client-server architecture. The architecture is divided into frontend, backend, database, AI processing, and RPA layers.

The frontend layer is developed using React.js with Vite and React Router. The frontend handles user interaction, navigation, form validation, and file uploading. The system also uses a

custom CSS design with dark mode support. Users can upload documents, search articles, review drafts, and manage workflow status through the frontend interface.

The backend layer is developed using Node.js and Express.js. The backend provides REST API services for uploading files, generating drafts, retrieving articles, updating workflow status, and handling database operations. The backend also handles document parsing and AI integration.

SQLite3 is used as the database for the system. The database stores article information, uploaded file details, workflow history, and user data. SQLite is lightweight and easy to configure for this project. The database is configured in WAL mode so that users and RPA bots can access the system smoothly at the same time.

The AI and processing layer uses Google Gemini API for summarizing and structuring operational documents. The system uses several parsing libraries to process different file types. These include pdf-parse for PDF files, mammoth for DOCX files, officeparser for PPTX files, mailparser for email files, and tesseract.js for image OCR processing.

The automation layer uses UiPath Studio. UiPath automates the process of reading files from folders and sending them into the system without manual work.

The system workflow starts when a user uploads a document. The backend extracts the text from the file. Then Google Gemini AI generates structured SOP content. After that, the draft is stored in the database. Reviewers can edit the article before changing the status from Draft to Reviewed and finally to Published.

3. API and JSON Structure

The backend provides REST APIs for CRUD operations and workflow management. These APIs allow the frontend and UiPath automation to communicate with the system.

One important API is the document upload API. The endpoint is POST /api/upload. This API accepts uploaded files using multipart form data. The backend extracts the text from the file

and stores the file information in the database. The API then returns the extracted text to the frontend.

Example response JSON:

```
{
  "success": true,
  "data": {
    "sourceFileId": 12,
    "fileName":
"scanner_reboot_guide.pdf",
    "extractedText": "Raw text extracted
from the document..."
  }
}
```

Another important API is the AI draft generation API. The endpoint is POST /api/draft. This API sends the extracted text to Google Gemini AI. The AI generates structured content such as title, summary, tags, steps, and warnings. The backend then creates a knowledge article in Draft status.

Example JSON generated by AI:

```
{
  "title": "Scanner Reboot Procedure",
  "summary": "Steps to reset the device after network failure.",
  "tags": ["scanner", "network", "hardware"],
  "steps": ["Hold power for 5s", "Select Reboot", "Reconnect to WiFi"],
  "warnings": ["Do not remove battery while restarting"]
}
```

The backend also provides APIs for article retrieval, status updates, and deletion. These include:

- GET /api/articles
- GET /api/articles/:id
- PUT /api/articles/:id

- PATCH /api/articles/:id/status
- DELETE /api/articles/:id

The system uses JSON as the main data format because JSON is simple and easy for frontend and backend communication.

4. RPA Workflow and Automation Explanation

The system uses UiPath Robotic Process Automation (RPA) to reduce manual work when processing large amounts of old operational documents.

The UiPath workflow starts by monitoring a specific folder called inputs/. When new files are added into the folder, the bot reads the files automatically. The bot uses a For Each File in Folder activity to process all files one by one.

After reading the file, the bot extracts the text content using file activities. Then the bot uses an HTTP Request activity to send a POST request to the backend API endpoint POST /api/draft/from-text. The backend processes the text and creates a draft article automatically.

The RPA workflow also includes logging and cleanup processes. When the API returns a successful response, the bot stores the processed file information into a log file called processedFiles.txt. Then the workflow continues with the next file.

The automation process helps DHL management process many operational documents quickly. Managers can place chat logs, SOP notes, or text documents into the monitored folder. Then the UiPath bot automatically uploads and converts them into knowledge base drafts for human review.

The RPA workflow also supports duplicate checking and error handling. The bot checks whether files were processed before. If duplicate files are detected, the bot skips the files. The workflow also uses Try-Catch activities for handling errors. When an error happens, the system records logs and continues processing other files.

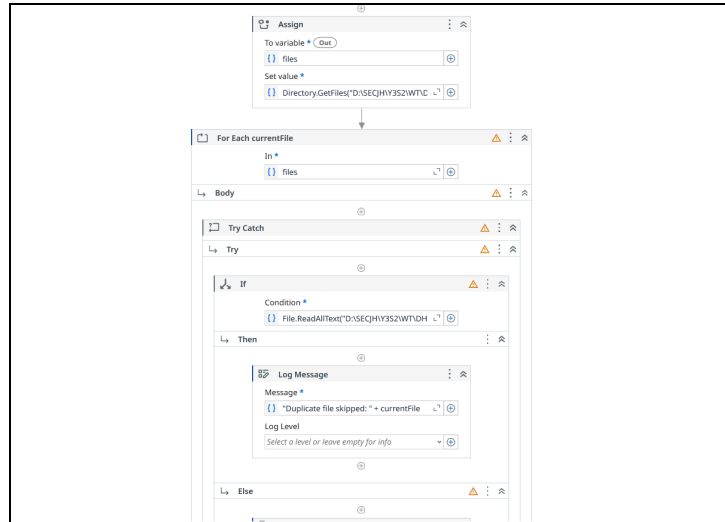


Figure 4.1 RPA Workflow of Duplicate Checking

5. User Interface Screenshots with Description

The Login Page is the first page users see when entering the system. This page allows users to access the AI-Powered Knowledge Base System by entering their full name, email address, and role.

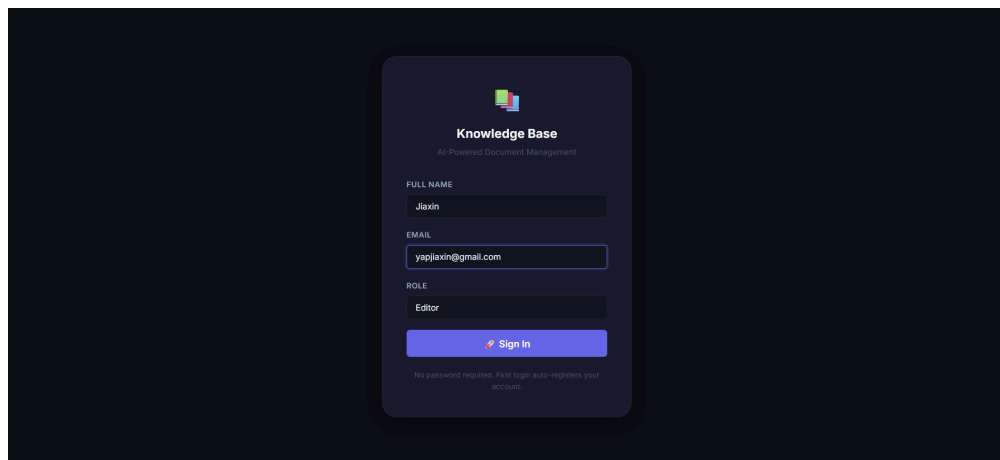


Figure 5.1 User Interface of Login Page

The Dashboard page is the main page of the system. This page displays the article list, search bar, and status filters. Users can search articles by keywords and filter articles using Draft, Reviewed, and Published status. Each article card displays the title, summary, version number, and workflow status.

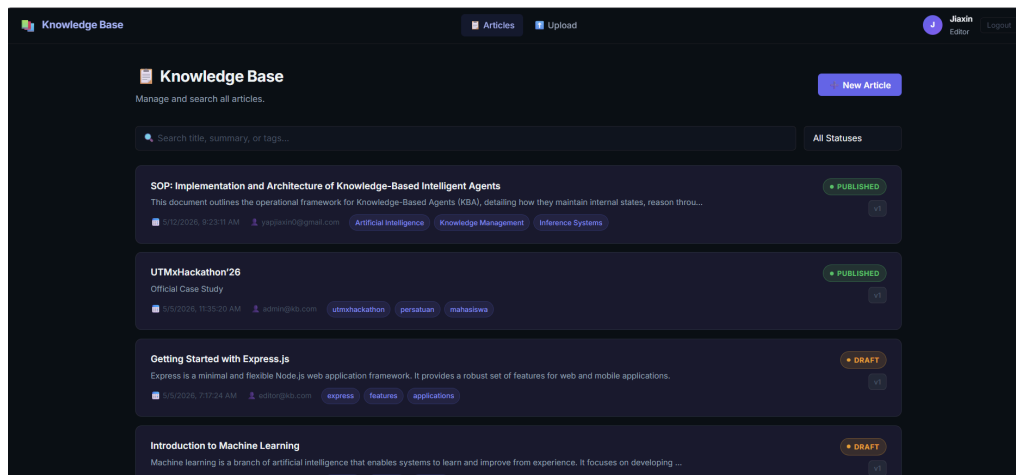


Figure 5.2 User Interface of Dashboard Page

The Upload Page allows users to upload files into the system. The page supports drag-and-drop uploading. Users can upload PDF, PPTX, DOCX, and image files. The page also allows users to paste raw operational text directly into the system.

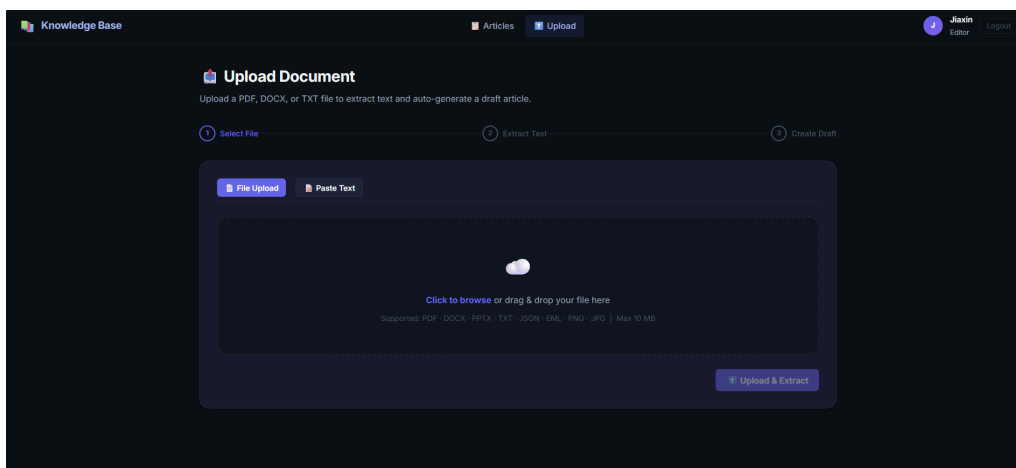


Figure 5.3 User Interface of Upload Page

The Extract Text Page displays a step-by-step workflow that includes file selection, text extraction, and draft article creation. After successful extraction, the system displays the extracted text preview inside a large text container. Users can review the extracted content before generating the draft article. This helps users verify that the uploaded file was processed correctly.

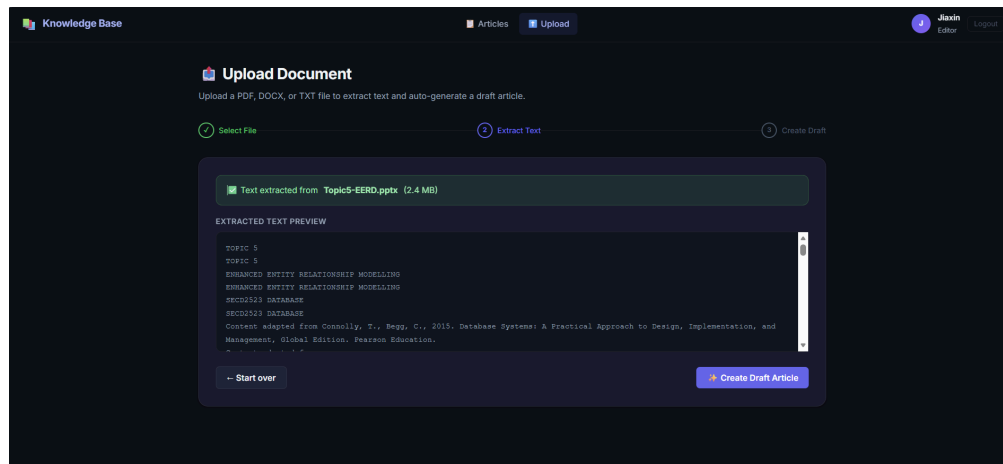
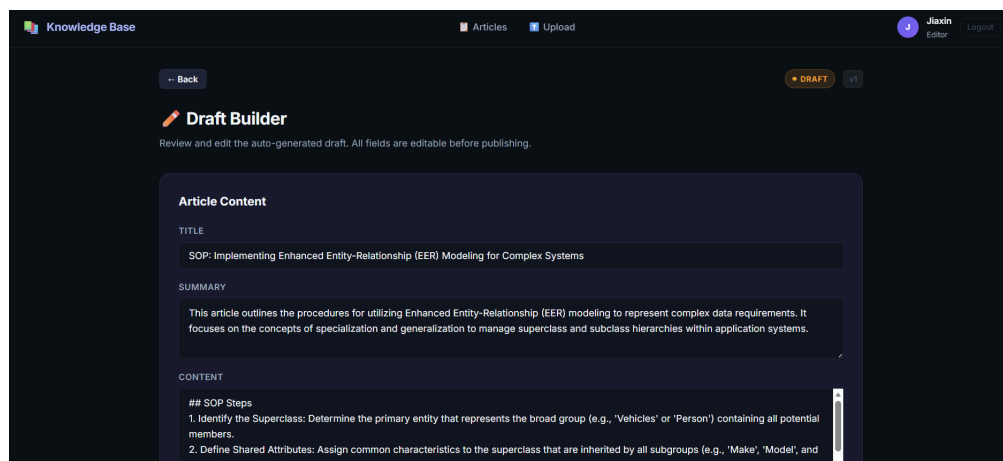


Figure 5.4 User Interface of Extract Text Page

The AI Draft Builder page displays the content generated by Google Gemini AI. The page shows the generated title, summary, tags, and SOP steps. Users can edit and improve the generated content before saving the article. The page also contains a discard button for removing unwanted drafts.



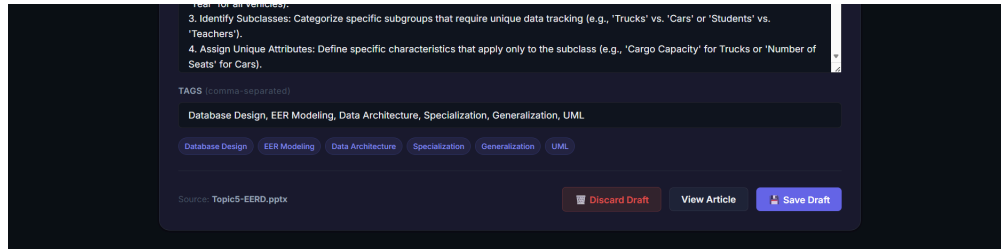


Figure 5.5 User Interface of AI Draft Builder Page

The Article Detail page displays the complete article information. Users can read the full SOP content and view the audit history. The page also contains workflow buttons for changing article status from Draft to Reviewed and Published. The audit history records who edited the article and when the status changed.

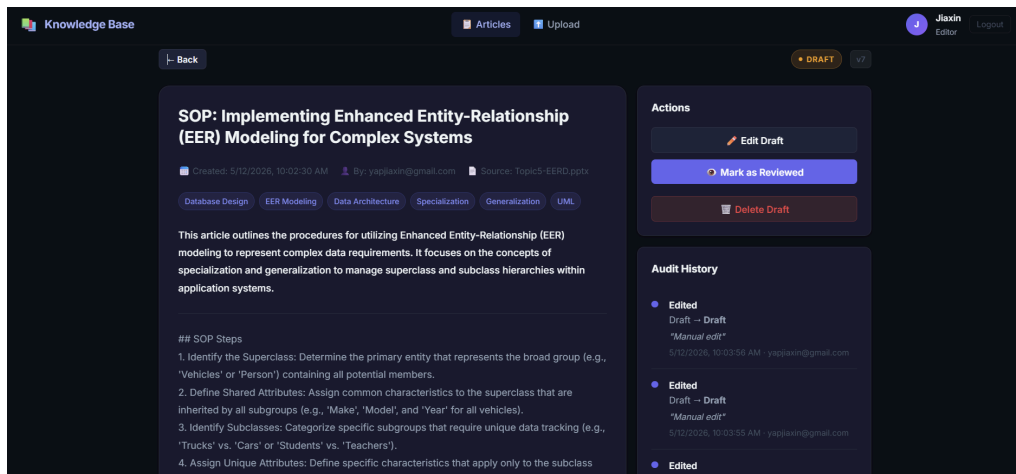


Figure 5.6 User Interface of Article Detail Page

6. User Manual

Before running the system, users need to install Node.js version 18 or above. Users also need a valid Google Gemini API key.

First, users need to configure the backend environment. Users should open the .env file inside the backend folder and add the Gemini API key together with the database path and server port.

Example configuration:

```
GEMINI_API_KEY=your_actual_api_key_here  
PORT=5000  
DB_PATH=./database/knowledge_base.db
```

Next, users need to start the backend server. Users should open a terminal and navigate to the backend folder. Then users need to install the required packages using `npm install`. After installation, users can run `npm run dev` to start the Express backend server.

After that, users need to start the frontend application. Users should open another terminal and navigate to the frontend folder. Then users need to install dependencies using `npm install` and run `npm run dev` to start the frontend server.

When both frontend and backend servers are running, users can open the browser and access the system through `http://localhost:5173`. Users can log into the system using an email and role selection. The system supports auto-registration for demonstration purposes.

To use the system, users can navigate to the Upload page and upload operational files or paste raw text. After clicking the upload button, the backend extracts the text and generates a draft article using Google Gemini AI. Users can review the generated SOP content and save the article. Finally, reviewers can update the workflow status from Draft to Reviewed and Published.